

Laboratory Manual

DEPARTMENT OF CSE (AI&ML)

PROGRAM - B.E CSE(AI ML)

FULL STACK WEB DEVELOPMENT LAB (22CAC11N)
SEMESTER - V

R-22A Regulation

Prepared by: Baby Lakshmi Prasanna, Falak Naaz

Verified by:



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE,
Accredited by NAAC with A++ Grade and Programs Accredited by NBA)
Chaitanya Bharathi Post, Gandipet, Kokapet (Vill.), Hyderabad, Ranga Reddy - 500 075, Telangana
www.cbit.ac.in



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE,
Accredited by NAAC with A++ Grade and Programs Accredited by NBA)
Chaitanya Bharathi Post, Gandipet, Kokapet (Vill.), Hyderabad, Ranga Reddy - 500 075, Telangana
www.cbit.ac.in

DEPARTMENT OF CSE(AI&ML)

Name of the Lab Course: FULL STACK DEVELOPMENT LAB (22CAC11N)

INDEX

Item	Page No
Institute Vision, Institute Mission, Quality Policy, Department Vision and Department Mission	
Program Educational Objectives (PEOs), Program Outcomes (POs) and Program Specific Outcomes (PSOs)	
Syllabus	
Course Introduction	
Assessment Procedure of CIE and SEE	
General Instructions for Laboratory Classes	

LABORATORY / PRACTICAL

Exp No	Laboratory / Practical	CO	BTL	Page No
1	Create a personal portfolio site with HTML5, CSS Grid, and Flexbox layout principles.	1	L6	1-3
2	Develop a JavaScript quiz app with a timer and scoring system using DOM manipulation.	1	L6	4-7
3	Build a weather forecast dashboard using JavaScript fetch API to consume a public weather API.	1	L6	8-12
4	Create a Responsive Class Timetable Using Bootstrap Grid System	2	L6	13-16
5	Create a multi-page site using React Router including home, about, and contact pages.	2	L6	19-22
6	Develop a basic ToDo app using React Hooks (useState, useEffect) and local storage.	3	L6	23-26
7	Build a contact form with validation using Angular 16 reactive forms (intro-level Angular usage).	3	L6	27-33
8	Create a RESTful API for book management using Express.js and test with Postman	4	L6	34-39

9	Implement MongoDB aggregation to summarize student grades, with schema design.	4	L6	40-44
10	Develop a full stack MERN app: Student registration form (React) connected to Express + MongoDB backend, deployed on local server.	5	L6	45-54



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE,
Accredited by NAAC with A++ Grade and Programs Accredited by NBA)
Chaitanya Bharathi Post, Gandipet, Kokapet (Vill.), Hyderabad, Ranga Reddy - 500 075, Telangana
www.cbit.ac.in

Vision of Institute

To be the Centre of Excellence in Technical Education and Research.

Mission of Institute

To address the Emerging needs through Quality Technical Education and
Advanced Research.

Quality Policy

CBIT imparts value based Technical Education and Training to meet the
requirements of students, Industry, Trade/
Profession, Research and
Development Organizations for Self-sustained Growth of Society.



CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE,
Accredited by NAAC with A++ Grade and Programs Accredited by NBA)
Chaitanya Bharathi Post, Gandipet, Kokapet (Vill.), Hyderabad, Ranga Reddy - 500 075, Telangana
www.cbit.ac.in

Department of CSE (AI&ML)

Vision of the Department

To produce professionals in artificial intelligence and machine learning through the best possible education, acquire international recognition as a destination, and advance society in exciting and creative ways.

Mission of the Department

The mission of the Artificial Intelligence and Machine Learning is to:

- 1. Impart rigorous training to generate knowledge through the state-of-the-art concepts and technologies in Artificial Intelligence and Machine Learning.*
- 2. Develop technical proficiency in students through creativity and leadership.*
- 3. Encourage lifelong learning, social responsibility, environmental conservation, and professional ethics.*
- 4. Establish centers of excellence in leading areas of computer and artificial intelligence disciplines.*

Program Educational Objectives (PEOs)

After the completion of the program, our:

- 1. Using a solid foundation in mathematical, scientific, engineering, and current computing principles, formulate, analyze, and resolve engineering issues.*
- 2. Apply artificial intelligence theory and concepts to analyze the requirements, realize technical specifications, and design engineering solutions.*
- 3. Through cross-disciplinary projects and a variety of professional activities, demonstrate technical proficiency, AI competency, and foster collaborative learning and a sense of teamwork.*
- 4. Provide graduates with solid knowledge, competence, and soft skills that will enable them to ethically contribute to societal demands and achieve sustainable advancement in emerging computer technologies through lifelong learning.*



CHAITANYABHARATHIINSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Osmania University, Approved by AICTE,
Accredited by NAAC with A++ Grade and Programs Accredited by NBA)

Chaitanya Bharathi Post, Gandipet, Kokapet (Vill.), Hyderabad, RangaReddy-500075, Telangana

www.cbit.ac.in

Department of CSE (AI&ML)

Program Outcomes (POs)

PO1. Apply the knowledge of mathematics, science, Engineering fundamentals, and an Engineering specialization to the solution of complex Engineering problems.

PO2.**Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1toWK4).

PO3.**Design/Development of solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet defined needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required, (WK5).

PO4.**Conduct investigations of complex problems:** Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. [WKB].

PO5.**Engineering Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2andWK6).

PO6.**The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.[WKI,WKS,andWK7].

PO7.**Ethics:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. **Individual and Collaborative Team Work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9.**Communication:** Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10. **Project Management and Finance** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11.Life-long learning: Recognize the need for, and have the preparation and ability for 1) independent and life-long learning ii) adaptability to new and emerging technologies and ill) critical thinking in the broadest context of technological change. (WKB)

Program Specific Outcomes (PSOs)

At the end of the program, Graduates

PSO1: Ability to evaluate and apply knowledge of data engineering, artificial intelligence, machine learning, and human cognition to real-world issues in order to solve potential challenges.

PSO2: Ability to acquire computational knowledge and project development abilities using novel tools and methodologies to tackle challenges in the fields related to Deep Learning, Machine learning, Artificial Intelligence

PSO3: Capacity to direct a team or firm that develops products and to use the knowledge learned to recognize actual research issues.

FULL STACK WEB DEVELOPMENT LAB

Instruction	3P Hours per week
Duration of SEE	3 Hours
SEE	50 Marks
CIE	50 Marks
Credits	1.5

Course Objectives

This course aims to:

- 1 Gain hands-on experience in designing modern web pages using HTML5, CSS3, and JavaScript.
- 2 Develop dynamic front-end applications using React and Angular fundamentals.
- 3 Integrate and test backend services using Node.js, Express, and MongoDB.

Course Outcomes:

Upon completion of this course, students will be able to:

- 1 Explain the structure and behavior of web interfaces
- 2 Apply ReactJS and Angular to develop front-end applications
- 3 Analyze external APIs and logic for interactive UI
- 4 Evaluate REST APIs using Node.js and Express
- 5 Design and implement MongoDB for full stack applications

CO-PO & PSO Articulation Matrix

PO/PSO CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO 1	3	2	2	1	2	1	-	2	2	1	2	2	2	2
CO 2	3	3	3	2	3	1	1	2	2	2	3	3	3	2
CO 3	3	3	3	3	3	1	1	2	2	2	3	3	3	3
CO 4	3	3	3	3	3	2	1	2	2	2	3	3	3	3
CO 5	3	3	3	3	3	2	1	2	2	2	3	3	3	3

LIST OF EXPERIMENTS

1. Create a personal portfolio site with HTML5, CSS Grid, and Flexbox layout principles.
2. Develop a JavaScript quiz app with a timer and scoring system using DOM manipulation.
3. Design a responsive blog template using Bootstrap 5 components and utilities.
4. Build a weather forecast dashboard using JavaScript fetch API to consume a public weather API.
5. Create a multi-page site using React Router including home, about, and contact pages.
6. Develop a basic ToDo app using React Hooks (useState, useEffect) and local storage.
7. Build a contact form with validation using Angular 16 reactive forms (intro-level Angular usage).
8. Create a RESTful API for book management using Express.js and test with Postman.
9. Implement MongoDB aggregation to summarize student grades, with schema design.
10. Develop a full stack MERN app: Student registration form (React) connected to Express + MongoDB backend, deployed on local server.

Text Books:

1. Jon Duckett, HTML and CSS: Design and Build Websites, Wiley India, 1st Edition, 2011.
2. Valeri Karpov, Professional JavaScript for Web Developers, Wiley, 4th Edition, 2020.
3. Ethan Brown, Web Development with Node and Express, O'Reilly Media, 1st Edition, 2014.

Suggested Reading:

1. Brad Traversy, Modern Full Stack Development: Using TypeScript, React, Node.js, Webpack, and Docker, Packt Publishing, 1st Edition, 2021.
2. Kristina Chodorow, MongoDB: The Definitive Guide, O'Reilly Media, 3rd Edition, 2019.

Web Resources:

1. <https://developer.mozilla.org>
2. <https://reactjs.org/docs>
3. <https://angular.io/docs>
4. <https://nodejs.org/en/docs>
5. <https://www.mongodb.com/docs/>
6. <https://getbootstrap.com/docs/5.0/getting-started/introduction/>

Course Introduction

This course introduces students to the core concepts of front-end and back-end web development using modern technologies such as HTML5, CSS3, JavaScript, Bootstrap, React, Angular, Express.js, Node.js, and MongoDB. Through practical exercises, students gain a strong foundation in designing and developing web applications that meet current industry standards.

The course focuses on building responsive user interfaces, implementing dynamic client-side functionality, and developing server-side applications with database connectivity. Students learn to use modern layout techniques, consume web APIs, create RESTful services, manage application state, perform CRUD operations, and integrate front-end and back-end components into complete web applications. Emphasis is placed on writing clean, efficient, and maintainable code while following best development practices.

By the end of the course, students will be able to design responsive websites, develop interactive web applications, implement secure and scalable REST APIs, and manage data using MongoDB. They will also gain hands-on experience in building full-stack MERN applications, strengthening their problem-solving abilities and preparing them for real-world web development projects and emerging industry requirements.

ASSESSMENT PROCEDURE AND AWARD OF CIE MARKS

Following is the subdivision for the internal marks (50) of the

Lab: i 20 marks for the lab internal tests

Two tests are to be conducted i.e one test after 1st cycle of experiments and second test after the second cycle. Average of two tests marks put together should be considered (20 maximum)

ii 30 marks for CIE

For the CIE 30 marks will be awarded based on the rubrics provided below.

This Rubrics are general guideline. Based on the lab type (programming or hardware) and complexity of the course Rubrics can be customized by the department in tune to program and course offered. Performance Indicators of the Rubrics also can be changed by the departments/Program based on the need.

RUBRICS

S. No	Parameter	Descriptors and Score				
		Outstanding	Good	Fair	Poor	Very Poor
1	Pre-Experiment Preparation work 5 M	Well prepared for the experimentation with a clear specifications, plan/design and additional information (5M)	prepared for the experimentation with clear specifications and plan/design (4M)	Adequately prepared for the experimentation without clear specifications and plan/design (3M)	Minimal preparation and without clear specifications and plan/design (2M)	Lacks preparation and without clear specifications and plan/design (1 M)
2	Experimentation (Problem Solving, Methodology of Conduction) 10 M	Student conducts experiment with all possible test cases in an optimized fashion.(10 M)	Student solves the problem and conducts experiments with all possible test cases (8M)	Student solves the problem and conducts experiment with few possible test cases with complexity (6M)	Student solves the problem with few test cases with complexity (4M)	Student fails to solve the problem (2M)
3	Post Experiment Analysis [Viva, Inference] 5M	Demonstrates the simulation/ findings /Hardware results Infers and answer all the Questions posed by Instructor (5M)	Demonstrates results and inference; Able to answer Few Questions posed by Instructor (4M)	Demonstrates results and inference; Unable to answer the Questions posed by Instructor. (3M)	Demonstrates Partial results and inference; Unable to answer the Questions posed by Instructor (2M)	Failed to Demonstrate results and inference; Unable to answer the Questions posed by Instructor (1M)
4	Report Writing 5M	Report meets all requirements and it is prepared in original and creative way to engage readers (5M)	Report with well-organized content, visuals, graphics, citations and references (4M)	Report is complete and adequate with poor grammar. (3M)	Report is complete, poor grammar and inadequate and failed to organize thoughts (2M)	Report is incomplete, unclear, poor grammar and inadequate (1M)

5	Conduct (Ethics, Safety, Team Work) 5M	Excellent team spirit, strictly follows ethics and safety precautions with good team	Follows the safety precautions, practices ethics and poor team work (4M)	Follows safety precautions and ethical practices and failed to exhibit teamwork.	Followed minimum safety precautions and ethical practices and failed to	Does not Follows safety precautions and ethical practices and failed to
---	---	---	---	---	---	---

S. N o	Parameter	Descriptors and Score				
		Outstanding	Good	Fair	Poor	Very Poor
		work (5M)		(3M)	exhibit team work. (2M)	exhibit team work. (1M)
TOTAL SCORE						

ASSESSMENT PROCEDURE AND AWARD OF SEE MARKS

S. No.	Parameter	Descriptors and Score				
		Outstanding	Good	Fair	Poor	Very Poor
1	Record 5 M	The content of all the experiments is well-organized, recorded the tables and neatly drawn graphs. Results and discussions are well presented. (5M)	The content of most of the experiments are well-organized, recorded the tables, neatly drawn graphs. Results and discussion are presented. (4M)	The content of the most of experiments are well-organized, recorded the tables, neatly drawn graphs. Results and discussion are not presented (3M)	The content of the few experiments is not well-organized, recorded the tables, neatly drawn graphs. Results and discussion are not presented (2M)	The content of the all the experiments is not well-organized, recorded the tables, neatly drawn graphs. Results and discussion are not presented (1M)
2	Write up about the experiment 10M	Presentation of the given experiment/program is very well organized with the required content, specifications, plan/procedure of the conduct and all the required additional information. (10M)	Presentation of the given experiment/program is organized with the required content, specifications, plan/procedure of the conduct. (8M)	Presentation of the given experiment/program is organized and is without clear specifications and plan/design. (6M)	Presentation of the given experiment/program is minimal with clear specifications and plan/design. (4M)	Presentation of the given experiment/program is minimum without clear specifications and plan/design. (2M)
3	Conduction of the experiment and observations 15M	Conducts experiment / Simulate the problem with proper connections / all possible test cases. All the possible observations are noted. (15M)	Conducts experiment / Simulate the problem with connections / with most of the test cases. Failed to note all observations (12M)	Conducts experiment / Simulate the problem with proper connections / less test cases. (9M)	Conducts experiment / Simulate the problem with improper connections / less test cases. (6M)	Failed to Conducts experiment / Simulate the problem with improper connections / less test cases. (3M)
4	Results & Analysis 10M	Demonstrates the experimental results with adequate analysis / simulation/ findings /obtained results /plotting the graphs. (10M)	Demonstrates the experimental results with required analysis / simulation/ findings /obtained results /plotting the graphs. (8M)	Demonstrates the experimental results, failed to maximum analysis / simulation/ findings. (6M)	Demonstrates the partial experimental results with least analysis / simulation/ findings. (4M)	Failed to Demonstrate the results and least analysis / simulation/ findings. (2M)
5	Viva-Voce 10M	Answers most of the questions with good analytical explanation. (10M)	Answers most of the questions with good explanation. (8M)	Answers only few of the questions with good explanation. (6M)	Answers only few of the questions with nominal explanation. (4M)	Failed to answer questions. (2M)
TOTAL SCORE						

GENERAL INSTRUCTIONS FOR LABORATORY CLASSES

DO'S

- 1 Without Prior permission do not enter into the Laboratory.
- 2 While entering into the LAB students should wear their ID cards.
- 3 The Students should come with proper uniform.
- 4 Students should sign in the LOGIN REGISTER before entering into the laboratory.
- 5 Students should come with observation and record note book to the laboratory.
- 6 Students should maintain silence inside the laboratory.
- 7 After completing the laboratory exercise, make sure to shutdown the system properly

DON'TS

- 1 Students bringing the bags inside the laboratory..
- 2 Students wearing slippers/shoes inside the laboratory.
- 3 Students using the computers in an improper way.
- 4 Students scribbling on the desk and mishandling the chairs.
- 5 Students using mobile phones inside the laboratory.
- 6 Students making noise inside the laboratory.

EXPERIMENT - 1

Create a personal portfolio site with HTML5, CSS Grid, and Flexbox layout principles.

OBJECTIVE OF THE EXPERIMENT:

To design a simple web page using HTML with headings, paragraphs, images, hyperlinks, and lists.

OUTCOME OF THE EXPERIMENT:

Students will be able to create a basic web page using HTML elements and understand the structure of an HTML document.

SYSTEM REQUIREMENTS:

1. HTML5-compliant Browser
2. Text Editor/IDE (e.g., VS Code, Notepad++)

ALGORITHM / PROCEDURE:

1. Open a text editor.
2. Create a new file and save it as `index.html`.
3. Write the basic HTML structure.
4. Add headings, paragraphs, lists, an image, and a hyperlink.
5. Save the file.
6. Open the file in a web browser to view the webpage.

PSEUDOCODE:

START

- ❖ Create an HTML document.
 - Add a title to the webpage.
- ❖ Display a heading.
- ❖ Display a paragraph.
- ❖ Create an unordered list.
- ❖ Insert an image.
- ❖ Add a hyperlink.
- ❖ Save the file.
- ❖ Open the webpage in a browser.

END

Sample Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>

  <h1>Welcome to HTML</h1>

  <p>
    HTML stands for HyperText Markup Language.
    It is used to create web pages.
  </p>

  <h2>My Hobbies</h2>

  <ul>
    <li>Reading</li>
    <li>Coding</li>
    <li>Playing Cricket</li>
  </ul>

  <br><br>

  <a href="https://www.google.com">
    Visit Google
  </a>

</body>
</html>
```

RESULTS:

A simple HTML webpage is created successfully displaying headings, paragraphs, a list, an image, and a hyperlink.

PRE LAB QUESTIONS:

1. What is HTML?
2. What is the purpose of HTML tags?
3. What is the difference between <head> and <body>?
4. What is the use of hyperlinks in HTML?

SAMPLE VIVA VOCE QUESTIONS:

1. What is HTML?
2. Explain the basic structure of an HTML document.
3. What is the purpose of the <title> tag?
4. What is the difference between ordered and unordered lists?
5. How do you insert an image in HTML?
6. What is the purpose of the <a> tag?
7. What is the use of the <body> tag?
8. Explain the difference between block-level and inline elements.
9. What are HTML attributes?
10. Why is HTML called a markup language?

EXPERIMENT - 2

Develop a JavaScript Quiz Application with Timer and Scoring System Using DOM Manipulation

OBJECTIVE OF THE EXPERIMENT:

To develop an interactive quiz application using JavaScript that includes a timer, scoring system, and DOM manipulation for dynamic question display and user interaction.

OUTCOME OF THE EXPERIMENT:

Students will be able to create a dynamic quiz application, manipulate HTML elements using the DOM, implement a countdown timer, and calculate scores using JavaScript.

SYSTEM REQUIREMENTS:

- HTML5-compliant Browser
- Text Editor/IDE (e.g., VS Code)

ALGORITHM / PROCEDURE:

1. Create the HTML structure for the quiz.
2. Style the application using CSS.
3. Define quiz questions and answers in JavaScript.
4. Display questions dynamically using DOM manipulation.
5. Start the timer when the quiz begins.
6. Check the selected answer and update the score.
7. Display the next question until all questions are completed.
8. Show the final score after the quiz ends or when the timer expires.

PSEUDOCODE:

START

Create quiz questions and answers.

Initialize score and timer.

Display the first question.

Start countdown timer.

WHILE questions remain AND timer > 0

 Accept user's answer.

 IF answer is correct

 Increase score.

 END IF

Display next question.
END WHILE

Display final score.

END

Sample Code:

HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>Quiz App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="quiz-container">
  <h2>JavaScript Quiz</h2>

  <div id="timer">Time: 30</div>

  <h3 id="question"></h3>

  <div id="options"></div>

  <button onclick="nextQuestion()">Next</button>

  <h3 id="result"></h3>
</div>

<script src="script.js"></script>

</body>
</html>
```

CSS

```
body{
  font-family: Arial, sans-serif;
  background:#f4f4f4;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}

.quiz-container{
  background:white;
  padding:20px;
  width:350px;
  border-radius:10px;
  box-shadow:0 0 10px gray;
}

button{
```

```

margin-top:15px;
padding:10px 20px;
background:#007bff;
color:white;
border:none;
cursor:pointer;
}

button:hover{
    background:#0056b3;
}

```

JavaScript:

```

const quiz = [
{
question:"Which language is used for web page interactivity?",
options:["HTML","CSS","JavaScript","Python"],
answer:"JavaScript"
},
{
question:"Which tag creates a heading?",
options:["<p>","<h1>","<img>","<a>"],
answer:"<h1>"
}
];

let index = 0;
let score = 0;
let time = 30;

const question = document.getElementById("question");
const options = document.getElementById("options");
const result = document.getElementById("result");

function loadQuestion(){
    question.innerHTML = quiz[index].question;
    options.innerHTML = "";

    quiz[index].options.forEach(option=>{
        options.innerHTML += `<input type="radio" name="ans" value="${option}"> ${option}<br>`;
    });
}

function nextQuestion(){
    const selected = document.querySelector('input[name="ans"]:checked');

    if(selected && selected.value === quiz[index].answer){
        score++;
    }

    index++;

    if(index < quiz.length){
        loadQuestion();
    }
    else{
        result.innerHTML = "Your Score: " + score + "/" + quiz.length;
        question.innerHTML = "";
        options.innerHTML = "";
    }
}

```

```

}

loadQuestion();

setInterval(function(){
    time--;
    document.getElementById("timer").innerHTML = "Time: " + time;

    if(time==0){
        result.innerHTML="Time Over! Score: "+score+"/"+quiz.length;
        question.innerHTML="";
        options.innerHTML="";
    }
},1000);

```

RESULTS:

A functional JavaScript quiz application with dynamic question display, countdown timer, and scoring system was developed successfully using DOM manipulation.

PRE LAB QUESTIONS:

1. What is DOM manipulation in JavaScript?
2. What is the purpose of `setInterval()`?
3. How can JavaScript access HTML elements?
4. What is the use of event handling in JavaScript?

SAMPLE VIVA VOCE QUESTIONS:

1. What is the Document Object Model (DOM)?
2. Explain the purpose of `getElementById()`.
3. What is the difference between `setInterval()` and `setTimeout()`?
4. How is the score calculated in the quiz application?
5. How does JavaScript dynamically create HTML elements?
6. What is event handling?
7. Why is a timer used in quiz applications?
8. What is the purpose of radio buttons in forms?
9. Explain the role of arrays in the quiz application.
10. How does JavaScript validate the selected answer?

EXPERIMENT - 3

Design a responsive blog template using Bootstrap 5 components and utilities

OBJECTIVE OF THE EXPERIMENT:

To design a responsive blog template using Bootstrap 5 components and utility classes for creating a modern, mobile-friendly web page.

OUTCOME OF THE EXPERIMENT:

Students will be able to create responsive web layouts using Bootstrap 5 components such as Navbar, Cards, Grid System, Buttons, and utility classes.

SYSTEM REQUIREMENTS:

- HTML5-compliant Browser
- Text Editor/IDE (e.g., VS Code)
- Bootstrap 5 CDN

ALGORITHM / PROCEDURE:

1. Create the basic HTML structure.
2. Link the Bootstrap 5 CSS and JavaScript CDN files.
3. Design a responsive navigation bar using Bootstrap Navbar.
4. Create blog post cards using Bootstrap Card components.
5. Arrange the cards using the Bootstrap Grid System.
6. Use Bootstrap utility classes for spacing, colors, and alignment.
7. Test the webpage on different screen sizes to verify responsiveness.

PSEUDOCODE:

START

Create an HTML document.

Include Bootstrap 5 CDN.

Create a responsive navigation bar.

Create a container.

Arrange blog cards using Bootstrap Grid.

Add images, titles, descriptions, and Read More buttons.

Apply Bootstrap utility classes.

Display responsive blog template.

END

Sample Code:

HTML:

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Bootstrap Blog</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">

<div class="container">

<a class="navbar-brand" href="#">My Blog</a>

<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#menu">

<span class="navbar-toggler-icon"></span>

</button>

<div class="collapse navbar-collapse" id="menu">

<ul class="navbar-nav ms-auto">

<li class="nav-item"><a class="nav-link" href="#">Home</a></li>

<li class="nav-item"><a class="nav-link" href="#">About</a></li>
```

```
<li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
```

```
</ul>
```

```
</div>
```

```
</div>
```

```
</nav>
```

```
<div class="container my-5">
```

```
<h2 class="text-center mb-4">Latest Blog Posts</h2>
```

```
<div class="row g-4">
```

```
<div class="col-md-4">
```

```
<div class="card h-100">
```

```

```

```
<div class="card-body">
```

```
<h5 class="card-title">Bootstrap Basics</h5>
```

```
<p class="card-text">
```

Learn how Bootstrap helps create responsive websites quickly.

```
</p>
```

```
<a href="#" class="btn btn-primary">Read More</a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<div class="card h-100">
```

```

```

```
<div class="card-body">  
<h5 class="card-title">Responsive Design</h5>  
<p class="card-text">  
Create layouts that work on desktops, tablets, and mobiles.
```

```
</p>  
<a href="#" class="btn btn-success">Read More</a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="col-md-4">
```

```
<div class="card h-100">
```

```

```

```
<div class="card-body">
```

```
<h5 class="card-title">Bootstrap Utilities</h5>
```

```
<p class="card-text">
```

```
Use spacing, typography, colors, and flex utilities effectively.
```

```
</p>
```

```
<a href="#" class="btn btn-danger">Read More</a>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
```

</body>

</html>

RESULTS:

A responsive blog template was successfully designed using Bootstrap 5 components, including the Navbar, Grid System, Cards, Buttons, and utility classes. The webpage adapts to different screen sizes and provides a clean, user-friendly layout.

PRE LAB QUESTIONS:

1. What is Bootstrap 5?
 2. What is the Bootstrap Grid System?
 3. What are Bootstrap utility classes?
 4. Why is responsive web design important?
-

SAMPLE VIVA VOCE QUESTIONS:

1. What is Bootstrap?
2. What are the advantages of using Bootstrap?
3. Explain the Bootstrap Grid System.
4. What is the purpose of the Navbar component?
5. What are Bootstrap Cards?
6. What are utility classes in Bootstrap?
7. Explain the use of **container** and **row** classes.
8. What is the difference between **container** and **container-fluid**?
9. How does Bootstrap make a webpage responsive?
10. What is the purpose of the Bootstrap CDN?

EXPERIMENT- 4

Build a weather forecast dashboard using JavaScript fetch API to consume a public weather API.

OBJECTIVE OF THE EXPERIMENT:

To develop a weather forecast dashboard using JavaScript Fetch API that retrieves real-time weather information from a public weather API and displays it dynamically.

OUTCOME OF THE EXPERIMENT:

Students will be able to fetch data from a public API using JavaScript Fetch API, parse JSON responses, and dynamically update web page content using DOM manipulation.

SYSTEM REQUIREMENTS:

- HTML5-compliant Browser
 - Text Editor/IDE (e.g., VS Code)
 - Internet Connection
 - Public Weather API (e.g., OpenWeatherMap API)
-

ALGORITHM / PROCEDURE:

1. Create the HTML structure for the weather dashboard.
2. Style the dashboard using CSS.
3. Create an input field to accept the city name.
4. Use the JavaScript Fetch API to send a request to the weather API.
5. Receive and parse the JSON response.
6. Display weather details such as temperature, humidity, and weather condition.
7. Handle invalid city names and network errors.
8. Test the application with different city names.

PSEUDOCODE:

START

Create HTML page.

Create input field and search button.

Read city name from user.

Send request to Weather API using Fetch API.

Receive JSON response.

```
IF city exists THEN
  Display temperature.
  Display humidity.
  Display weather description.
ELSE
  Display error message.
END IF

END
```

Sample Code:

HTML:

```
<!DOCTYPE html>
<html>
<head>
  <title>Weather Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h2>Weather Forecast Dashboard</h2>

    <input type="text" id="city" placeholder="Enter City
      Name">

    <button onclick="getWeather()">Search</button>

    <div id="weather"></div>

  </div>

  <script src="script.js"></script>

</body>
</html>
```

CSS:

```
body{
  font-family: Arial, sans-serif;
  background:#eaf4fc;
  display:flex;
  justify-content:center;
  align-items:center;
  height:100vh;
}

.container{
  background:white;
  padding:20px;
  border-radius:10px;
  box-shadow:0 0 10px gray;
  text-align:center;
```

```

    width:320px;
  }

input{
  width:90%;
  padding:10px;
  margin:10px 0;
}

button{
  padding:10px 20px;
  background:#007BFF;
  color:white;
  border:none;
  cursor:pointer;
}

button:hover{
  background:#0056b3;
}

#weather{
  margin-top:20px;
  font-size:18px;
}

```

javascript

```

function getWeather(){

const city=document.getElementById("city").value;

const apiKey="YOUR_API_KEY";

const url=`https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=${apiKey}&units=metric`;

fetch(url)

.then(response=>response.json())

.then(data=>{

if(data.cod===200){

document.getElementById("weather").innerHTML=`
<h3>${data.name}</h3>
<p>Temperature : ${data.main.temp} °C</p>
<p>Humidity : ${data.main.humidity}%</p>
<p>Weather : ${data.weather[0].description}</p>
`;

}

else{

document.getElementById("weather").innerHTML="City not found.";

}

})

.catch(error=>{

```

```
document.getElementById("weather").innerHTML="Unable to fetch weather data.";
});
}
```

RESULTS:

A weather forecast dashboard was successfully developed using the JavaScript Fetch API. The application retrieves weather information from a public weather API and dynamically displays the current temperature, humidity, and weather conditions based on the entered city.

PRE LAB QUESTIONS:

1. What is the Fetch API in JavaScript?
2. What is a REST API?
3. What is JSON?
4. Why is an API key required for weather services?

SAMPLE VIVA VOCE QUESTIONS:

1. What is the Fetch API?
2. What is the purpose of `fetch()` in JavaScript?
3. What is JSON and why is it used?
4. What is a public API?
5. Explain asynchronous programming in JavaScript.
6. What is the role of Promises in Fetch API?
7. Why is an API key required?
8. How do you handle errors while fetching data?
9. What is DOM manipulation?
10. What are the advantages of using APIs in web development?

CASE STUDY EXPERIMENT

Create a Timer Component to Start, Pause, and Reset Using ReactJS

OBJECTIVE OF THE EXPERIMENT: To develop a timer component using ReactJS that allows users to start, pause, and reset the timer.

OUTCOME OF THE EXPERIMENT: Students will understand React state management and event handling.

SYSTEM REQUIREMENTS:

- Node.js and npm
- ReactJS environment
- Browser with developer tools

ALGORITHM / PROCEDURE:

1. Create a React component for the timer.
2. Use React state to manage the timer's value.
3. Implement functions for starting, pausing, and resetting the timer.
4. Bind the functions to buttons.

PSEUDOCODE:

```
START  
INITIALIZE state for time and running status  
CREATE functions to handle start, pause, and reset actions  
USE setInterval to update time when running  
CLEAR interval on pause or reset  
DISPLAY time and buttons  
END
```

Sample Code:

```
import React, { useState, useEffect } from 'react';  
  
function Timer() {  
  const [time, setTime] = useState(0);  
  const [isRunning, setIsRunning] = useState(false);  
  
  useEffect(() => {  
    let timer;  
    if (isRunning) {  
      timer = setInterval(() => setTime((prevTime) => prevTime + 1), 1000);  
    } else {
```

```

clearInterval(timer);
}
return () => clearInterval(timer);
}, [isRunning]);

return (
<div>
<h1>Timer: {time}s</h1>
<button onClick={() => setIsRunning(true)}>Start</button>
<button onClick={() => setIsRunning(false)}>Pause</button>
<button onClick={() => { setIsRunning(false); setTime(0); }}>Reset</button>
</div>
);
}

export default Timer;

```

RESULTS:

A functional timer component that can start, pause, and reset.

PRE LAB QUESTIONS:

1. How does the useEffect hook work in React?
2. What are the benefits of using React state?

SAMPLE VIVA VOCE QUESTIONS:

1. Explain the lifecycle of a React component.
2. How can you optimize a timer component in React?

EXPERIMENT - 5

Create a Multi-Page Site Using React Router (Home, About, and Contact Pages)

OBJECTIVE OF THE EXPERIMENT:

To develop a multi-page React application using **React Router** that allows navigation between **Home**, **About**, and **Contact** pages without reloading the browser.

OUTCOME OF THE EXPERIMENT:

Students will understand:

- React Router for client-side routing.
- Navigation between multiple pages.
- Creating reusable React components.
- Basic React application structure.

SYSTEM REQUIREMENTS:

- Node.js and npm
 - ReactJS environment
 - React Router (`react-router-dom`)
 - Browser with Developer Tools
-

ALGORITHM / PROCEDURE:

1. Create a new React application.
2. Install the React Router package.
3. Create three components:
 - Home
 - About
 - Contact
4. Configure routing using `BrowserRouter`, `Routes`, and `Route`.
5. Add navigation links using `Link`.
6. Run the application and verify navigation between pages.

PSEUDOCODE:

START

Create Home component

Create About component

Create Contact component

Wrap application with BrowserRouter

Create navigation menu

Home Link

About Link

Contact Link

Define Routes

"/" → Home

"/about" → About

"/contact" → Contact

Display corresponding page when user clicks a link

END

```
import React from "react";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";
```

App.js

```
function App() {
  return (
    <BrowserRouter>
      <div>
        <nav>
          <Link to="/">Home</Link> | {" "}
          <Link to="/about">About</Link> | {" "}
          <Link to="/contact">Contact</Link>
        </nav>

        <hr />
      </div>
    </BrowserRouter>
  );
}
```

```
    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>
  </div>
</BrowserRouter>
);
}

export default App;
```

```
import React from "react";
```

Home.js

```
function About() {
  return (
    <div>
      <h1>About Page</h1>
      <p>This is the About page.</p>
    </div>
  );
}

export default About;
```

```
import React from "react";
```

```
function Contact() {
  return (
    <div>
      <h1>Contact Page</h1>
      <p>You can contact us at contact@example.com</p>
    </div>
  );
}

export default Contact;
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Install React Router

npm install react-router-dom

RESULTS:

A functional React application with multiple pages was successfully developed. Users can navigate between the **Home**, **About**, and **Contact** pages using React Router without refreshing the browser.

PRE LAB QUESTIONS:

1. What is a React Router?
2. What is the purpose of BrowserRouter?

SAMPLE VIVA VOCE QUESTIONS:

1. What is the difference between Link and <a> tag?
2. What are Routes and Route in React Router?
3. Why do we use BrowserRouter?
4. What is client-side routing?

EXPERIMENT - 6

Develop a basic ToDo app using React Hooks (useState, useEffect) and local storage.

OBJECTIVE OF THE EXPERIMENT:

To develop a React-based ToDo application using React Hooks (useState and useEffect) that allows users to add, delete, and manage tasks while storing the task list in the browser's Local Storage so that data persists even after refreshing the page.

OUTCOME OF THE EXPERIMENT:

Students will understand:

- React Hooks (useState and useEffect).
- State management in React.
- Browser Local Storage for persistent data.
- Handling user events in React.
- Creating reusable React components.

SYSTEM REQUIREMENTS:

- Node.js and npm
- ReactJS Environment
- Visual Studio Code
- Web Browser (Chrome/Edge/Firefox)

ALGORITHM / PROCEDURE:

1. Create a new React application using Vite or Create React App.
2. Create a ToDo component.
3. Initialize task list using useState.
4. Load previously saved tasks from Local Storage using useEffect.
5. Allow users to enter a new task.
6. Add the task to the list.
7. Save updated tasks to Local Storage whenever the task list changes.
8. Display all tasks.
9. Delete a task when the Delete button is clicked.
10. Run the application and verify task persistence after page refresh.

PSEUDOCODE:

START

Initialize tasks using useState

Load tasks from Local Storage using useEffect

Display input textbox

Display Add button

IF Add button clicked THEN

 Add task to task list

ENDIF

Save updated task list to Local Storage

Display all tasks

IF Delete button clicked THEN

 Remove selected task

ENDIF

Update Local Storage

END

App.jsx

```
import React, { useState, useEffect } from "react";
```

```
function App() {
```

```
  const [task, setTask] = useState("");
```

```
  const [tasks, setTasks] = useState(() => {  
    const savedTasks = localStorage.getItem("tasks");  
    return savedTasks ? JSON.parse(savedTasks) : [];  
  });
```

```
  useEffect(() => {  
    localStorage.setItem("tasks", JSON.stringify(tasks));  
  }, [tasks]);
```

```
  const addTask = () => {  
    if (task.trim() === "") return;
```

```
    setTasks([...tasks, task]);  
    setTask("");  
  };
```

```

const deleteTask = (index) => {
  const updatedTasks = tasks.filter((_, i) => i !== index);
  setTasks(updatedTasks);
};

return (
  <div style={{ textAlign: "center", marginTop: "30px" }}>

    <h2>React ToDo App</h2>

    <input
      type="text"
      placeholder="Enter Task"
      value={task}
      onChange={(e) => setTask(e.target.value)}
    />

    <button onClick={addTask}>Add</button>

    <ul>
      {tasks.map((item, index) => (
        <li key={index}>
          {item}
          <button onClick={() => deleteTask(index)}>
            Delete
          </button>
        </li>
      ))}
    </ul>

  </div>
);
}

export default App;

```

Main.jsx

```

import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

Install React

npm create vite@latest todo-app

Choose

React

JavaScript

Move into project

cd todo-app

Install dependencies

npm install

Run the project

npm run dev

RESULTS:

A functional React ToDo application was successfully developed using **useState** and **useEffect** hooks. Users can add and delete tasks, and the task list is stored in **Local Storage**, allowing it to persist even after refreshing the browser.

PRE LAB QUESTIONS:

1. What is the purpose of the **useState** hook in React?
2. What is Local Storage, and why is it used?

SAMPLE VIVA VOCE QUESTIONS:

1. What is the difference between **useState** and **useEffect**?
2. Why do we use Local Storage in a React application?
3. What is the purpose of the dependency array in **useEffect**?
4. How do you update the state in React?
5. What happens if Local Storage is not used in this application?
6. How does React re-render components when the state changes?
7. What is the purpose of the **map()** function in this program?
8. Why is the **key** attribute required while displaying lists in React?
9. What is event handling in React?
10. Explain the workflow of this ToDo application.

EXPERIMENT - 7

Build a contact form with validation using Angular 16 reactive forms (intro-level Angular usage).

OBJECTIVE OF THE EXPERIMENT

To develop a Contact Form using Angular 16 Reactive Forms with built-in form validation. The experiment enables students to create a responsive form, validate user inputs, display validation messages, and understand the basics of Angular reactive forms.

OUTCOME OF THE EXPERIMENT

After completing this experiment, students will be able to:

1. Understand the concept of Angular Reactive Forms.
2. Create a contact form using FormGroup and FormControl.
3. Apply built-in validators such as Required, Email, and Minimum Length.
4. Display validation error messages dynamically.
5. Handle form submission using Angular.
6. Improve user experience through client-side validation.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB (Minimum)
- Hard Disk: 2 GB Free Space

Software Requirements

- Operating System: Windows 10/11 / Linux / macOS
- Node.js (Latest LTS)
- Angular CLI 16
- Visual Studio Code
- Google Chrome or Microsoft Edge
- npm (Node Package Manager)

ALGORITHM

1. Start.
2. Create a new Angular project.
3. Import the ReactiveFormsModule.
4. Create a Contact Component.
5. Define a FormGroup with the required controls.
6. Apply validators to each control.
7. Design the HTML form using Angular directives.
8. Display validation messages for invalid inputs.
9. Check whether the form is valid.
10. If valid, display a success message.
11. Otherwise, display validation errors.
12. Stop.

PSEUDOCODE:

BEGIN

Create ContactForm

Add fields

 Name

 Email

 Subject

 Message

Apply Validators

 Name -> Required

 Email -> Required + Email

 Subject -> Required

 Message -> Required + Minimum Length

WHEN Submit Button is Clicked

 IF Form is Valid THEN

 Display "Form Submitted Successfully"

 ELSE

Display Validation Messages

ENDIF

END

[app.module.ts](#)

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
import { ReactiveFormsModule } from '@angular/forms';
```

```
import { AppComponent } from './app.component';
```

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [  
    BrowserModule,  
    ReactiveFormsModule  
  ],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

app.component.ts

```
import { Component } from '@angular/core';  
import { FormGroup, FormControl, Validators } from '@angular/forms';
```

```
@Component({  
  selector: 'app-root',
```

```

    templateUrl: './app.component.html'
  })
  export class AppComponent {

    contactForm = new FormGroup({
      name: new FormControl("", Validators.required),
      email: new FormControl("", [
        Validators.required,
        Validators.email
      ]),
      subject: new FormControl("", Validators.required),
      message: new FormControl("", [
        Validators.required,
        Validators.minLength(10)
      ])
    });

    onSubmit() {
      if (this.contactForm.valid) {
        alert("Form Submitted Successfully");
        console.log(this.contactForm.value);
      }
    }
  }
}

```

App.component.html

```

<h2>Contact Form</h2>

<form [formGroup]="contactForm" (ngSubmit)="onSubmit()">

  <label>Name</label><br>
  <input type="text" formControlName="name">

  <div *ngIf="contactForm.controls.name.invalid && contactForm.controls.name.touched">
    Name is required.
  </div>

  <br><br>

```

```
<label>Email</label><br>
<input type="email" formControlName="email">

<div *ngIf="contactForm.controls.email.invalid && contactForm.controls.email.touched">
  Enter a valid email.
</div>

<br><br>

<label>Subject</label><br>
<input type="text" formControlName="subject">

<div *ngIf="contactForm.controls.subject.invalid && contactForm.controls.subject.touched">
  Subject is required.
</div>

<br><br>

<label>Message</label><br>

<textarea rows="5" formControlName="message"></textarea>

<div *ngIf="contactForm.controls.message.invalid && contactForm.controls.message.touched">
  Message should contain at least 10 characters.
</div>

<br><br>

<button type="submit" [disabled]="contactForm.invalid">
  Submit
</button>

</form>
```

OUTPUT

Contact Form

Name : _____

Email : _____

Subject : _____

Message :

[Submit]

If fields are empty:

Name is required.
Enter a valid email.
Subject is required.
Message should contain at least 10 characters.

If valid:

Form Submitted Successfully

Pre-Lab Questions

1. What are Reactive Forms in Angular, and why are they used?
2. What is the purpose of the ReactiveFormsModule in Angular?
3. What is the difference between a FormGroup and a FormControl?
4. Name any four built-in validators available in Angular Reactive Forms.
5. What is the purpose of the FormControlName directive?

Pre-Viva Questions

1. What is the difference between Template-driven Forms and Reactive Forms in Angular?
2. What is the role of the `Validators.required` validator?
3. How do you check whether a Reactive Form is valid before submitting it?
4. What is the purpose of the `(ngSubmit)` event in Angular forms?
5. Why is form validation important in web applications?

Experiment-8

Create a RESTful API for Book Management using Express.js and Test with Postman

OBJECTIVE OF THE EXPERIMENT

To develop a **RESTful API** for managing book records using **Express.js** and to test the API endpoints using **Postman**. This experiment helps students understand CRUD (Create, Read, Update, Delete) operations, HTTP methods, and API testing.

OUTCOME OF THE EXPERIMENT

After completing this experiment, students will be able to:

1. Understand the concept of RESTful APIs.
2. Develop REST API endpoints using Express.js.
3. Implement CRUD operations for book management.
4. Handle HTTP requests and responses.
5. Test API endpoints using Postman.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB (Minimum)
- Hard Disk: 2 GB Free Space

Software Requirements

- Windows 10/11 / Linux / macOS
- Node.js (Latest LTS)
- Express.js
- Visual Studio Code
- Postman
- Google Chrome or Microsoft Edge

ALGORITHM

1. Start.
2. Create a Node.js project.
3. Install Express.js.
4. Import Express and create an application.
5. Create an array to store book records.
6. Define REST API routes:
 - GET – Retrieve all books.
 - POST – Add a new book.
 - PUT – Update an existing book.
 - DELETE – Remove a book.
7. Start the Express server.
8. Open Postman.
9. Test each API endpoint.
10. Verify the responses.
11. Stop.

PSEUDOCODE

BEGIN

Create Express Server

Initialize Books Array

GET /books

Return all books

POST /books

Add new book

PUT /books/:id

Update book details

DELETE /books/:id

Delete book

Start Server

Test APIs using Postman

END

server.js

```
const express = require('express');
const app = express();

app.use(express.json());

let books = [
  { id: 1, title: "JavaScript Basics", author: "John" },
  { id: 2, title: "Node.js Guide", author: "David" }
];

// GET all books
app.get('/books', (req, res) => {
  res.json(books);
});

// POST a new book
app.post('/books', (req, res) => {
  const book = req.body;
  books.push(book);
  res.json({
    message: "Book Added Successfully",
    book
  });
});

// PUT update a book
app.put('/books/:id', (req, res) => {

  const id = parseInt(req.params.id);

  const book = books.find(b => b.id === id);

  if (book) {
    book.title = req.body.title;
    book.author = req.body.author;
```

```

    res.json({
      message: "Book Updated Successfully",
      book
    });
  }
  else {
    res.status(404).json({
      message: "Book Not Found"
    });
  }
});

// DELETE a book
app.delete('/books/:id', (req, res) => {

  const id = parseInt(req.params.id);

  books = books.filter(book => book.id !== id);

  res.json({
    message: "Book Deleted Successfully"
  });
});

// Start Server
app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});

```

POSTMAN TESTING

HTTP Method	URL	Description
GET	http://localhost:3000/books	Retrieve all books
POST	http://localhost:3000/books	Add a new book
PUT	http://localhost:3000/books /1	Update book with ID 1

DELETE	http://localhost:3000/books/1	Delete book with ID 1
--------	-------------------------------	-----------------------

Sample JSON for POST/PUT

```
{  
  "id": 3,  
  "title": "Express.js",  
  "author": "James"  
}
```

OUTPUT

GET Request

```
[  
  {  
    "id": 1,  
    "title": "JavaScript Basics",  
    "author": "John"  
  },  
  {  
    "id": 2,  
    "title": "Node.js Guide",  
    "author": "David"  
  }  
]
```

POST Request

```
{  
  "message": "Book Added Successfully",  
  "book": {  
    "id": 3,  
    "title": "Express.js",  
    "author": "James"  
  }  
}
```

PUT Request

```
{  
  "message": "Book Updated Successfully",  
  "book": {
```

```
"id": 1,  
"title": "Advanced JavaScript",  
"author": "John"  
}  
}
```

DELETE Request

```
{  
  "message": "Book Deleted Successfully"  
}
```

PRE-LAB QUESTIONS

1. What is a RESTful API?
2. What is the purpose of Express.js in Node.js applications?
3. Name the four HTTP methods used for CRUD operations.
4. What is the purpose of `express.json()` middleware?
5. What is Postman, and why is it used?

PRE-VIVA QUESTIONS

1. What is the difference between GET and POST requests?
2. Why do we use route parameters such as `:id` in Express.js?
3. What is the purpose of `req.body` in an Express application?
4. How do you start an Express.js server?
5. How do you test REST API endpoints using Postman?

EXPERIMENT-9

Implement MongoDB Aggregation to Summarize Student Grades with Schema Design

OBJECTIVE OF THE EXPERIMENT

To design a **MongoDB schema** for storing student grade information and implement **aggregation pipeline operations** to summarize student performance, including total marks, average marks, and overall grade statistics.

OUTCOME OF THE EXPERIMENT

After completing this experiment, students will be able to:

1. Design a MongoDB schema for student grade management.
2. Insert student documents into a MongoDB collection.
3. Apply MongoDB aggregation pipeline operators.
4. Calculate total and average marks using aggregation.
5. Analyze student performance using grouped summary data.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB (Minimum)
- Hard Disk: 2 GB Free Space

Software Requirements

- Windows 10/11 / Linux / macOS
- Node.js (Latest LTS)
- MongoDB Community Server
- MongoDB Compass
- Visual Studio Code
- Mongo Shell (mongosh)

ALGORITHM

1. Start.
2. Create a MongoDB database.
3. Create a **students** collection.
4. Design the student schema with fields such as Roll Number, Name, Department, and Marks.
5. Insert student records into the collection.
6. Apply the MongoDB aggregation pipeline.

7. Group student records by department.
8. Calculate total marks, average marks, and student count.
9. Display the summarized results.
10. Stop.

PSEUDOCODE

BEGIN

Create students collection

Insert student documents

Apply Aggregation Pipeline

Group documents by Department

Calculate

Student Count

Total Marks

Average Marks

Display Summary

END

SAMPLE CODE

Student Schema (Sample Document)

```
db.students.insertMany([
{
  rollNo: 101,
  name: "Alice",
  department: "CSE",
  marks: 85
},
{
  rollNo: 102,
```

```

    name: "Bob",
    department: "CSE",
    marks: 78
  },
  {
    rollNo: 103,
    name: "Charlie",
    department: "ECE",
    marks: 90
  },
  {
    rollNo: 104,
    name: "David",
    department: "ECE",
    marks: 82
  },
  {
    rollNo: 105,
    name: "Emma",
    department: "CSE",
    marks: 95
  }
]);

```

MongoDB Aggregation Query

```

db.students.aggregate([
  {
    $group: {
      _id: "$department",
      totalStudents: { $sum: 1 },
      totalMarks: { $sum: "$marks" },
      averageMarks: { $avg: "$marks" },
      highestMarks: { $max: "$marks" },
      lowestMarks: { $min: "$marks" }
    }
  }
]);

```

SAMPLE OUTPUT

```
[
{
  "_id" : "CSE",
  "totalStudents" : 3,
  "totalMarks" : 258,
  "averageMarks" : 86,
  "highestMarks" : 95,
  "lowestMarks" : 78
},
{
  "_id" : "ECE",
  "totalStudents" : 2,
  "totalMarks" : 172,
  "averageMarks" : 86,
  "highestMarks" : 90,
  "lowestMarks" : 82
}
]
```

SCHEMA DESIGN

Field Name	Data Type	Description
rollNo	Number	Unique roll number of the student
name	String	Student name
department	String	Department name
marks	Number	Marks obtained by the student

PRE-LAB QUESTIONS

1. What is MongoDB, and how is it different from relational databases?
2. What is a document in MongoDB?
3. What is the purpose of a schema in MongoDB?
4. What is the MongoDB aggregation framework?

5. Name any three aggregation operators used in MongoDB.

PRE-VIVA QUESTIONS (5)

1. What is the purpose of the \$group stage in MongoDB aggregation?
2. What is the difference between \$sum and \$avg operators?
3. How does MongoDB store documents internally?
4. What is the advantage of using aggregation over simple queries?
5. How can MongoDB Compass be used to view collections and aggregation results?

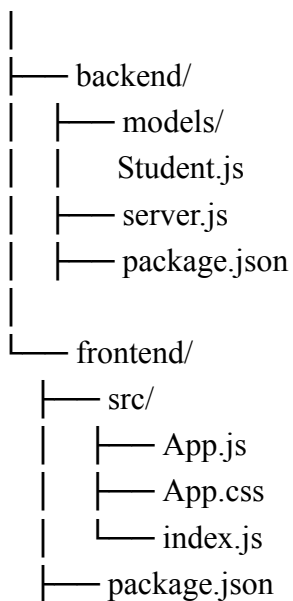
Develop a full stack MERN app: Student registration form (React) connected to Express + MongoDB backend, deployed on a local server.

Software Requirements

- Visual Studio Code
- Node.js (Latest LTS)
- MongoDB Community Server
- MongoDB Compass (Optional)
- React
- Express.js
- Postman (Optional)

Project Structure

StudentRegistrationMERN/



STEP 1: Create Backend

Open VS Code Terminal

```
mkdir StudentRegistrationMERN
cd StudentRegistrationMERN
```

```
mkdir backend
cd backend
npm init -y
```

Install packages

```
npm install express mongoose cors
npm install nodemon --save-dev
```

```
server.js
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect("mongodb://127.0.0.1:27017/studentdb")
.then(()=>console.log("MongoDB Connected"))
.catch(err=>console.log(err));

const StudentSchema = new mongoose.Schema({

  name:String,
  email:String,
  mobile:String,
  course:String

});

const Student = mongoose.model("Student",StudentSchema);

app.post("/register",async(req,res)=>{

  try{

    const student = new Student(req.body);
    await student.save();

    res.json({
      message:"Student Registered Successfully"
    });
  }
```

```
}

catch(err){

  res.status(500).json(err);

}

});

app.get("/students",async(req,res)=>{

  const students=await Student.find();

  res.json(students);

});

app.listen(5000,()=>{

  console.log("Server Running on Port 5000");

});
```

STEP 2: Start Backend

```
npx nodemon server.js
```

Output

```
MongoDB Connected
Server Running on Port 5000
```

STEP 3: Create React Project

Open another terminal.

```
cd ..
```

```
npx create-react-app frontend
```

```
cd frontend
```

Install Axios

```
npm install axios
```

```
App.js
```

```
import React,{useState} from "react";  
import axios from "axios";  
import "./App.css";
```

```
function App(){
```

```
  const [student,setStudent]=useState({
```

```
    name:"",  
    email:"",  
    mobile:"",  
    course:""
```

```
  });
```

```
  const handleChange=(e)=>{
```

```
    setStudent({
```

```
      ...student,  
      [e.target.name]:e.target.value
```

```
    });
```

```
  };
```

```
  const handleSubmit=async(e)=>{
```

```
    e.preventDefault();
```

```
    await axios.post("http://localhost:5000/register",student);
```

```
alert("Student Registered Successfully");
```

```
setStudent({
```

```
  name:"",
```

```
  email:"",
```

```
  mobile:"",
```

```
  course:""
```

```
});
```

```
};
```

```
return(
```

```
<div className="container">
```

```
<h2>Student Registration Form</h2>
```

```
<form onSubmit={handleSubmit}>
```

```
<input
```

```
  type="text"
```

```
  name="name"
```

```
  placeholder="Enter Name"
```

```
  value={student.name}
```

```
  onChange={handleChange}
```

```
  required
```

```
<input
```

```
  type="email"
```

```
  name="email"
```

```
  placeholder="Enter Email"
```

```
  value={student.email}
```

```
  onChange={handleChange}
```

```
  required
```

```
<input
```

```
  type="text"
```

```
name="mobile"
placeholder="Enter Mobile"
value={student.mobile}
onChange={handleChange}
required
/>
```

```
<input
type="text"
name="course"
placeholder="Enter Course"
value={student.course}
onChange={handleChange}
required
/>
```

```
<button type="submit">
```

```
Register
```

```
</button>
```

```
</form>
```

```
</div>
```

```
);
```

```
}
```

```
export default App;
```

```
App.css
```

```
body{
```

```
font-family:Arial;
```

```
background:#f4f4f4;
```

```
}
```

```
.container{  
  
width:400px;  
margin:40px auto;  
background:white;  
padding:20px;  
border-radius:10px;  
box-shadow:0 0 10px gray;
```

```
}
```

```
h2{  
  
text-align:center;
```

```
}
```

```
input{  
  
width:100%;  
padding:10px;  
margin:10px 0;
```

```
}
```

```
button{  
  
width:100%;  
padding:12px;  
background:green;  
color:white;  
font-size:18px;  
border:none;  
cursor:pointer;
```

```
}
```

```
button:hover{  
  
background:darkgreen;
```

```
}
```

STEP 4: Start React

```
npm start
```

Runs on

```
http://localhost:3000
```

STEP 5: Start MongoDB

If MongoDB is installed as a Windows service, simply ensure the **MongoDB** service is running (check **Services** in Windows). If not installed as a service, start it manually:

```
mongod
```

MongoDB will listen on:

```
mongodb://127.0.0.1:27017
```

STEP 6: Run Application

Terminal 1

```
cd backend
```

```
npx nodemon server.js
```

Output

```
MongoDB Connected
```

```
Server Running on Port 5000
```

Terminal 2

```
cd frontend
```

```
npm start
```

Output

```
Compiled Successfully
```

```
Local:
```

http://localhost:3000

STEP 7: Test

Open

http://localhost:3000

Enter

Name

Email

Mobile

Course

Click

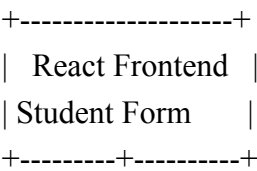
Register

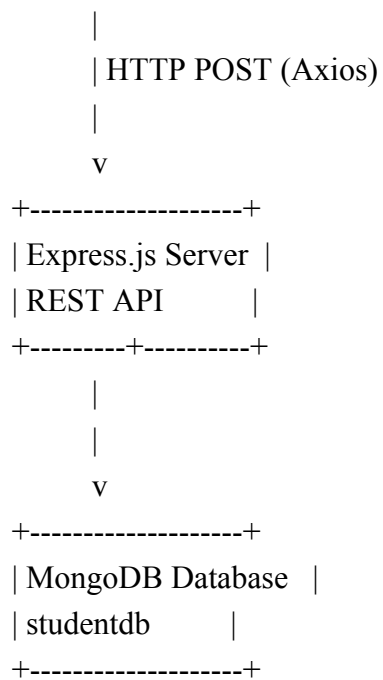
The data is saved into the **studentdb** database in the **students** collection.

MongoDB Collection Example

```
{
  "_id": "6879abcd1234567890ef1234",
  "name": "Lakshmi",
  "email": "lakshmi@gmail.com",
  "mobile": "9876543210",
  "course": "B.Tech CSE",
  "__v": 0
}
```

Project Workflow





GAP ANALYSIS BY THE COURSE FACULTY / COORDINATOR

.....
.....
.....

Faculty name & Signature with date

HoD/BoS Chairman

remarks:

.....
.....
.....

HoD/BoS chairman Signature with date